

Linear search

```
#include <stdio.h>
```

```
// Linear Search
```

```
int linearSearch(int arr[], int n, int key) {  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == key) return i;  
    }  
    return -1;  
}
```

```
// Binary Search (Requires sorted array)
```

```
int binarySearch(int arr[], int low, int high, int key) {  
    while (low <= high) {  
        int mid = low + (high - low) / 2;  
        if (arr[mid] == key) return mid;  
        if (arr[mid] < key) low = mid + 1;  
        else high = mid - 1;  
    }  
    return -1;  
}
```

```
int main() {  
    int arr[] = {10, 20, 30, 40, 50, 60};  
    int n = sizeof(arr) / sizeof(arr[0]);  
    printf("Linear Search (20): %d\n", linearSearch(arr, n, 20));  
    printf("Binary Search (50): %d\n", binarySearch(arr, 0, n - 1, 50));  
    return 0;  
}
```

Stack implementation aarray

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack[MAX];
```

```
int top = -1;
```

```
void push(int data) {  
    if (top == MAX - 1) {  
        printf("Stack Overflow!\n");  
        return;  
    }  
    stack[++top] = data;  
    printf("%d pushed.\n", data);  
}
```

```
int pop() {
```

```

    if (top == -1) {
        printf("Stack Underflow!\n");
        return -1;
    }
    return stack[top--];
}

```

```

int main() {
    push(10);
    push(20);
    printf("Popped: %d\n", pop());
    return 0;
}

```

Queue using array

```

#include <stdio.h>
#define MAX_SIZE 5

```

```

int queue[MAX_SIZE];
int front = -1, rear = -1;

```

```

void enqueue(int data) {
    if (rear == MAX_SIZE - 1) {
        printf("Queue Overflow!\n");
        return;
    }
    if (front == -1) front = 0;
    queue[++rear] = data;
    printf("%d enqueued.\n", data);
}

```

```

int dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow!\n");
        front = rear = -1; // Reset
        return -1;
    }
    return queue[front++];
}

```

```

int main() {
    enqueue(100);
    enqueue(200);
    printf("Dequeued: %d\n", dequeue());
    return 0;
}

```

Linked list

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* head = NULL;

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// 4.a. Insert at beginning (Stack Push / Queue Dequeue point)
void insertAtBeginning(int data) {
    struct Node* newNode = createNode(data);
    newNode->next = head;
    head = newNode;
}

// 4.b. Insert at end (Queue Enqueue point)
void insertAtEnd(int data) {
    struct Node* newNode = createNode(data);
    if (head == NULL) {
        head = newNode;
        return;
    }
    struct Node* current = head;
    while (current->next != NULL) current = current->next;
    current->next = newNode;
}

// 4.c. Insert at any position
void insertAtPosition(int data, int position) {
    if (position <= 1) { insertAtBeginning(data); return; }
    struct Node* newNode = createNode(data);
    struct Node* current = head;
    for (int i = 1; current != NULL && i < position - 1; i++) {
        current = current->next;
    }
    if (current == NULL) { printf("Invalid position.\n"); return; }
    newNode->next = current->next;
```

```
    current->next = newNode;
}
```

// 4.d. Delete at beginning (Stack Pop / Queue Dequeue point)

```
void deleteAtBeginning() {
    if (head == NULL) { printf("List empty.\n"); return; }
    struct Node* temp = head;
    head = head->next;
    free(temp);
}
```

// 4.e. Delete element 'x' and Delete at end

```
void deleteX(int x) {
    struct Node *current = head, *prev = NULL;
    if (current != NULL && current->data == x) { // x is head
        head = current->next; free(current); return;
    }
    while (current != NULL && current->data != x) {
        prev = current;
        current = current->next;
    }
    if (current == NULL) return; // Not found
    prev->next = current->next;
    free(current);
}
```

```
void displayList() {
    struct Node* current = head;
    printf("List: ");
    while (current != NULL) {
        printf("%d -> ", current->data);
        current = current->next;
    }
    printf("NULL\n");
}
```

```
int main() {
    printf("--- Linked List Operations ---\n");
    insertAtBeginning(10);
    insertAtEnd(30);
    insertAtPosition(20, 2);
    displayList(); // 10 -> 20 -> 30 -> NULL

    printf("--- Stack (LIFO) using LL (Push/Pop = Beg/Beg) ---\n");
    // insertAtBeginning is Push, deleteAtBeginning is Pop

    printf("--- Queue (FIFO) using LL (Enqueue = End, Dequeue = Beg) ---\n");
    // insertAtEnd is Enqueue, deleteAtBeginning is Dequeue
}
```

```
    return 0;
}
```

Best and bull shit

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node *left, *right;
};
```

```
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}
```

// 8.a. Insert a node in BST

```
struct Node* insertBST(struct Node* node, int data) {
    if (node == NULL) return createNode(data);
    if (data < node->data) node->left = insertBST(node->left, data);
    else if (data > node->data) node->right = insertBST(node->right, data);
    return node;
}
```

// 7.a. Count nodes

```
int countNodes(struct Node* node) {
    if (node == NULL) return 0;
    return 1 + countNodes(node->left) + countNodes(node->right);
}
```

// 7.b. Count leaf nodes

```
int countLeafNodes(struct Node* node) {
    if (node == NULL) return 0;
    if (node->left == NULL && node->right == NULL) return 1;
    return countLeafNodes(node->left) + countLeafNodes(node->right);
}
```

// 7.c. Count non-leaf nodes

```
int countNonLeafNodes(struct Node* node) {
    if (node == NULL || (node->left == NULL && node->right == NULL)) return 0;
    return 1 + countNonLeafNodes(node->left) + countNonLeafNodes(node->right);
}
```

// 7.d. Determine height

```

int treeHeight(struct Node* node) {
    if (node == NULL) return -1;
    int leftHeight = treeHeight(node->left);
    int rightHeight = treeHeight(node->right);
    return (leftHeight > rightHeight ? leftHeight : rightHeight) + 1;
}

// 8.b. Find minimum node from BST
struct Node* findMinNode(struct Node* node) {
    struct Node* current = node;
    while (current && current->left != NULL) current = current->left;
    return current;
}

// 8.c. Traversal (Inorder)
void inorder(struct Node* node) {
    if (node != NULL) {
        inorder(node->left);
        printf("%d ", node->data);
        inorder(node->right);
    }
}

int main() {
    struct Node* root = NULL;
    root = insertBST(root, 50);
    insertBST(root, 30);
    insertBST(root, 70);
    insertBST(root, 20);

    printf("Inorder Traversal: ");
    inorder(root);
    printf("\nTotal Nodes: %d\n", countNodes(root));
    return 0;
}

```

Heap and

```

#include <stdio.h>
#include <stdlib.h>

#define MAX_HEAP_SIZE 100
int heap[MAX_HEAP_SIZE];
int size = 0;

void swap(int* a, int* b) { int temp = *a; *a = *b; *b = temp; }

void heapifyUp(int i) {

```

```

    int parent = (i - 1) / 2;
    while (i > 0 && heap[i] > heap[parent]) {
        swap(&heap[i], &heap[parent]);
        i = parent;
        parent = (i - 1) / 2;
    }
}

void heapifyDown(int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < size && heap[left] > heap[largest]) largest = left;
    if (right < size && heap[right] > heap[largest]) largest = right;

    if (largest != i) {
        swap(&heap[i], &heap[largest]);
        heapifyDown(largest);
    }
}

// 9.a. Insert root element (actually inserts anywhere, then heapifies)
void insertMaxHeap(int data) {
    if (size >= MAX_HEAP_SIZE) { printf("Heap Overflow!\n"); return; }
    heap[size] = data;
    size++;
    heapifyUp(size - 1);
}

// 9.b. Delete root element (max element)
int deleteRootFromMaxHeap() {
    if (size <= 0) { printf("Heap empty.\n"); return -1; }
    int max_val = heap[0];
    heap[0] = heap[size - 1];
    size--;
    heapifyDown(0);
    return max_val;
}

int main() {
    insertMaxHeap(10);
    insertMaxHeap(50);
    insertMaxHeap(20);
    printf("Deleted Max Element: %d\n", deleteRootFromMaxHeap());
    return 0;
}

```

Sorting algorithm

```
#include <stdio.h>
```

```
void swap(int* a, int* b) { int temp = *a; *a = *b; *b = temp; }
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) printf("%d ", arr[i]);
    printf("\n");
}
```

// 10.b. Selection Sort

```
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) min_idx = j;
        }
        if (min_idx != i) swap(&arr[min_idx], &arr[i]);
    }
}
```

// 10.c. Bubble Sort

```
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) swap(&arr[j], &arr[j + 1]);
        }
    }
}
```

// 10.d. Insertion Sort

```
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

// 10.e. Quick Sort

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
```

```

        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

```

// 10.f. Merge Sort

```

void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

```

// 10.g. Heap Sort Helper

```

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest]) largest = left;

```

```

    if (right < n && arr[right] > arr[largest]) largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}
// 10.g. Heap Sort
void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}

```

```

int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = 6;
    // Example: selectionSort(arr, n);
    quickSort(arr, 0, n - 1);
    printf("Sorted Array: ");
    printArray(arr, n);
    return 0;
}

```

Bfs and dfs

```

#include <stdio.h>
#include <stdlib.h>

#define MAX_VERTICES 50

// Adjacency List Structures
struct Node {
    int dest;
    struct Node* next;
};
struct Graph {
    int numVertices;
    struct Node* adjLists[MAX_VERTICES];
    int visited[MAX_VERTICES];
};

struct Node* createNode(int dest) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->dest = dest;
    newNode->next = NULL;
}

```

```

    return newNode;
}

struct Graph* createGraph(int vertices) {
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    graph->numVertices = vertices;
    for (int i = 0; i < vertices; i++) {
        graph->adjLists[i] = NULL;
        graph->visited[i] = 0;
    }
    return graph;
}

void addEdge(struct Graph* graph, int src, int dest) {
    // For undirected graph
    struct Node* newNode = createNode(dest);
    newNode->next = graph->adjLists[src];
    graph->adjLists[src] = newNode;

    newNode = createNode(src);
    newNode->next = graph->adjLists[dest];
    graph->adjLists[dest] = newNode;
}

void resetVisited(struct Graph* graph) {
    for (int i = 0; i < graph->numVertices; i++) graph->visited[i] = 0;
}

// --- 11.a. Breadth First Search (BFS) ---
int queue[MAX_VERTICES];
int front = -1, rear = -1;
void enqueue(int item) {
    if (front == -1) front = 0;
    queue[++rear] = item;
}
int dequeue() {
    if (front == -1) return -1;
    int item = queue[front++];
    if (front > rear) front = rear = -1;
    return item;
}

void BFS(struct Graph* graph, int startVertex) {
    resetVisited(graph);
    enqueue(startVertex);
    graph->visited[startVertex] = 1;

    printf("BFS Traversal: ");

```

```

while (front != -1) {
    int currentVertex = dequeue();
    printf("%d ", currentVertex);

    struct Node* temp = graph->adjLists[currentVertex];
    while (temp) {
        int adjVertex = temp->dest;
        if (graph->visited[adjVertex] == 0) {
            graph->visited[adjVertex] = 1;
            enqueue(adjVertex);
        }
        temp = temp->next;
    }
}
}

```

// --- 11.b. Depth First Search (DFS) ---

```

void DFS(struct Graph* graph, int vertex) {
    graph->visited[vertex] = 1;
    printf("%d ", vertex);

    struct Node* temp = graph->adjLists[vertex];
    while (temp) {
        int connectedVertex = temp->dest;
        if (graph->visited[connectedVertex] == 0) {
            DFS(graph, connectedVertex);
        }
        temp = temp->next;
    }
}
}

```

```

int main() {
    struct Graph* graph = createGraph(5);
    addEdge(graph, 0, 1);
    addEdge(graph, 0, 4);
    addEdge(graph, 1, 2);
    addEdge(graph, 2, 3);
    addEdge(graph, 3, 4);

    BFS(graph, 0); // Starting from 0
    printf("\n");

    resetVisited(graph);
    printf("DFS Traversal: ");
    DFS(graph, 0); // Starting from 0
    printf("\n");
    return 0;
}

```

